

Roping Manager â Technical Overview

Last updated: 18 February 2026

1. Purpose & Scope

This document captures the current functionality, technology stack, and backend interfaces of the Roping Manager desktop app so that engineers, operators, and integrators can reason about every feature offered today. It consolidates the React/Tauri frontend, the Rust/SQLite backend, and supporting tooling into a single technical reference for onboarding and audit purposes.

2. Technology Stack

| Layer | Key Technologies | Evidence |

| --- | --- | --- |

| Desktop container | Tauri 2 with 'tauri-plugin-dialog' & 'tauri-plugin-opener'; Rust 2021 edition

| '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/Cargo.toml:1-47' |

| Frontend | React 19, TypeScript 5.8, Vite 7, TailwindCSS 3.4, Radix UI primitives, Lucide icons, 'react-big-calendar', 'recharts', Storybook 9 | '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/package.json:6-70' |

| Data & IPC | SQLite (via 'sqlx'), Tokio async runtime, '@tauri-apps/api' invocations, 'rust_xlsxwriter' for Excel exports | '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/Cargo.toml:20-44', '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/lib/api.ts:1-235' |

| Tooling & QA | npm scripts for dev/build/storybook, Tauri CLI, Vitest + Playwright, Chromatic | '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/package.json:6-70' |

3. Architecture Overview

- **App shell & navigation.** 'App' holds a 'Sidebar'-driven state machine that swaps view-level components inside a flex layout, while Sonner toasts stay mounted globally.

'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/App.tsx:1-79' couples navigation IDs with view components, and '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/Sidebar.tsx:18-83' renders the accessible nav rail.

- **IPC boundary.** All data mutations and reads flow through thin wrappers in '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/lib/api.ts:1-235', which map camelCase props into the snake_case payloads Rust expects.

- **Backend runtime.** 'run()' in '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/src/lib.rs:2353-2443' boots SQLite, applies migrations, registers every '#[tauri::command]', and exposes Excel export utilities backed by 'rust_xlsxwriter'.

- **State helpers.** Hooks such as '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/hooks/useTeams.ts:1-89' and '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/hooks/useRopers.ts:1-159' normalize backend rows and enforce lock-state validation before writing.

4. Functional Coverage (Frontend)

4.1 Dashboard & Navigation

- **Series-centric overview.** 'Dashboard' orchestrates the entire workflow: it loads series, stats, and activity logs, coordinates modal CRUD operations, and conditionally renders 'EventsView' or 'EventDetails' panes. '/Users/emilianoneaves/Documents/RMT/roping-manager-

tauri/src/components/Dashboard.tsx:64-200' covers state, fetching, and CRUD handlers.

- **Metrics sidebar.** **'MetricsPanel'** summarizes counts (series, events, teams, pot, progress) using cards and progress bars fed by `'get_dashboard_stats'`, see `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/MetricsPanel.tsx:1-94'`.
- **Recent activity widget.** **'RecentActivity'** renders audit log snippets with action-based iconography to keep dashboard users situationally aware. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/RecentActivity.tsx:1-77'`.

4.2 Event Lifecycle Workspace

- **Series event browser.** **'EventsView'** combines search, status filters, card/table toggles, and modals for new/edit/delete actions while enforcing admin PIN gates. It loads only events that belong to the selected series and wires duplication, export, draw, capture, standings, and payoff actions. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/EventsView.tsx:23-200'`.
- **Series insights.** **'SeriesOverviewCard'** and **'InsightsPanel'** give aggregated counts, pot projections, and activity feed for the open series, helping producers prioritize. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/SeriesOverviewCard.tsx:1-60'`, `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/InsightsPanel.tsx:1-120'`.
- **Event workspace layout.** **'EventDetails'** presents breadcrumb navigation, an **'EventMetricsCard'**, and tabbed tooling for teams, draw, capture, standings, payoffs, and exports. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/EventDetails.tsx:1-160'`.
 - **Teams Tab.** Dragnet for roster health, filtering, sorting, and creation flows (manual or auto-balanced pairing) while honoring rating caps and lock status. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/TeamsTab.tsx:1-210'` integrates `'useTeams'`/`'useRopers'` plus hard-delete + rebuild routines.
 - **Draw Tab.** Generates entire rounds or individual rounds with shuffling, reseeding, and validation checks via `'generate_draw'` and `'generate_draw_batch'`. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/DrawTab.tsx:1-150'`.
 - **Capture Tab.** **'CaptureRunsTab'** embeds a chronometer, keyboard shortcuts, manual override mode, PIN-protected overwrites, and auto-locking of events after the first saved run (`'update_event_status'`). `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/CaptureRunsTab.tsx:59-520'`.
 - **Standings Tab.** Real-time aggregation with filters, podium cards, and Excel export hook (`'export_event_to_excel'`). `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/StandingsTab.tsx:1-200'`.
 - **Payoffs Tab.** Preset-driven prize allocation, CRUD over payoff rules, and payout visualization backed by `'list_payoff_rules'` and `'get_payout_breakdown'`. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/PayoffsTab.tsx:1-200'`.
 - **Export Tab.** Sheet selection UI that pipes user choices into the backend exporter via Tauri's dialog plugin. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/ExportTab.tsx:1-180'`.

4.3 Cross-Series Operational Views

- **Global calendar.** **'EventsCalendar'** renders every event on `'react-big-calendar'`, color-codes statuses, and offers inline creation via `'NewEventModal'`. `'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/EventsCalendar.tsx:1-180'`.

- **Teams Management (multi-event).** Enables organizers to switch series/event filters, auto-create balanced rosters, and hard-reset teams when needed. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/TeamsManagement.tsx:39-200`.
- **Ropers Management.** Full CRUD, CSV/XLSX import, and bulk deletion of ropers using the 'xlsx' library and the 'useRopers' hook. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/RopersManagement.tsx:22-200`.
- **Capture Management.** Guides operators through selecting series/events, summarises readiness KPIs, and launches the focused 'EventCaptureView'. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/CaptureManagement.tsx:1-140`.
- **Event Capture fullscreen.** Provides the same capture workflow outside tabs, with keyboard shortcuts, timer controls, and direct Excel export. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/EventCaptureView.tsx:1-200`.
- **Results & Analytics.** 'ResultsManagement' and 'PayoffsManagement' expose cross-event filters, payout summaries, and export buttons so staff can audit outcomes even after events close. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/ResultsManagement.tsx:1-200`, `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/PayoffsManagement.tsx:1-200`.
- **Export Center.** 'ExportManagement' wraps recurring export settings, include/exclude toggles, and a mock history table for operational traceability. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/ExportManagement.tsx:1-200`.
- **Activity Log.** Infinite-scroll audit viewer with tone-coded actions and 'get_recent_activity' pagination. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/ActivityLogView.tsx:1-124`.
- **Settings.** Multi-tab preference surface for user profile, storage, backups, appearance, notifications, and locale toggles. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/components/SettingsManagement.tsx:1-200`.

5. Backend Command Surface

The Tauri backend exposes every functional area through dedicated commands registered in `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/src/lib.rs:2394-2443`. Detailed behaviors mirror the Spanish documentation in `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/BACKEND_DOCUMENTATION.md:93-260`.

| Domain | Commands & Highlights |

| --- | --- |

| Health | 'health_check' for DB readiness. |

| Series | 'list_series', 'create_series', 'update_series', 'delete_series' (with cascade safeguards). |

| Events | 'list_events', 'list_all_events_raw', 'create_event', 'update_event', 'update_event_status', 'duplicate_event', 'delete_event', 'lock_event'.

`/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/BACKEND_DOCUMENTATION.md:128-158` |

| Teams | 'list_teams', 'create_team', 'update_team', 'delete_team', 'hard_delete_teams_for_event' enforce uniqueness and lock rules. `/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/BACKEND_DOCUMENTATION.md:172-190` |

| Ropers | 'list_ropers', 'create_roper', 'update_roper', 'delete_roper', 'delete_all_ropers'.

'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/BACKEND_DOCUMENTATION.md:193-207' |
| Draw & Runs | 'generate_draw', 'generate_draw_batch', 'get_draw', 'save_run', 'get_runs', 'get_runs_expanded' maintain run-order integrity before capture.
'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/BACKEND_DOCUMENTATION.md:160-220' |
Standings & Analytics	'get_standings', 'get_dashboard_stats', 'get_series_logs', 'get_recent_activity'. '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/BACKEND_DOCUMENTATION.md:210-233'
Payoffs	'list_payoff_rules', 'create_payoff_rule', 'delete_payoff_rule', 'get_payout_breakdown' (see struct implementations at '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/src/lib.rs:583-750').
Export	'export_event_to_excel' builds multi-sheet XLSX files (overview, teams, run order, standings, payoffs, logs) with 'rust_xlsxwriter'. '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/src/lib.rs:2150-2341'

IPC Reliability Considerations

- 'log_audit' is invoked after every mutating command to populate 'audit_log', though frontend still treats activity feed as informational because auth is not yet enforced.
- Commands rely on 'ensure_event_unlocked' internally, so UI attempts will surface backend errors if someone bypasses the locking UX.

6. Data Model Snapshot

Based on migrations summarized in '/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/BACKEND_DOCUMENTATION.md:60-90':

- **Core tables:** 'series', 'event', 'team', 'roper', 'run', 'draw', 'payoff_rule', 'payoff', 'audit_log'.
- **Security scaffolding:** 'app_user', 'role', 'user_role', and 'license_info' exist for future auth/licensing, yet no frontend flows consume them today.
- **Integrity rules:** Rounded counts, status enums, and triggers ensure rating caps and permissible values ('status', 'rounds', 'level').

7. Frontend Backend Integration Patterns

- **API wrappers.** '@tauri-apps/api' s 'invoke' is centralized so payload quirks (snake_case IDs, status normalization) are patched once instead of per view.
'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/lib/api.ts:21-175'.
- **State hooks.** 'useTeams' and 'useRopers' debounce refetches, normalize mixed-case column names, and block writes when events are locked, ensuring UI logic mirrors backend constraints.
'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/hooks/useTeams.ts:4-88',
'/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src/hooks/useRopers.ts:18-159'.
- **File operations.** Excel imports/exports leverage the shared 'xlsx' npm package on the frontend ('RopersManagement') and 'rust_xlsxwriter' on the backend for reports, giving parity between desk and operator workflows.

8. Build, Test, and Ops

- **Local dev.** 'npm run dev' starts Vite, and 'npm run tauri dev' spawns the desktop shell.

`‘/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/package.json:6-13’`.

- **Quality gates.** Storybook (`‘npm run storybook’`), Vitest + Playwright combos, and Chromatic snapshots are wired for component regression coverage. `‘/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/package.json:6-70’`.

- **Database tooling.** `‘sqlx-cli’` plus the `‘docs/dev_reset_db.md’` playbook (not modified here) allow wiping or reseeding local SQLite instances when QA scenarios demand a clean slate.

9. Observations & Recommended Next Steps

Borrowing from the backend TODOs `‘/Users/emilianoneaves/Documents/RMT/roping-manager-tauri/src-tauri/BACKEND_DOCUMENTATION.md:232-249’` and current frontend gaps:

1. **Authentication & Authorization.** Implement Argon2-backed login flows and gate IPC commands per `‘user_role’` to protect the audit trail.
2. **Audit completeness.** Frontend mutations should await confirmation that `‘log_audit’` succeeded (or retry) so Activity Log remains exhaustive.
3. **Integration tests.** Add Rust integration specs for `‘create_team’`, `‘generate_draw’`, `‘save_run’`, and `‘get_standings’`, plus Vitest suites for hooks.
4. **Type sharing.** Generate a shared TypeScript schema (e.g., via `‘ts-rs’`) so frontend payload types stay in sync with backend structs.
5. **Licensing hooks.** If device licensing is required, wire `‘license_info’` validations into app startup and expose user feedback in Settings.